

Introduction to jQuery

Most popular server-side programming languages

© W3Techs.com	usage	change since 1 July 2026
1. PHP	70.2%	-0.6%
2. JavaScript	7.2%	+0.5%
3. Ruby	7.0%	+0.2%
4. Java	5.4%	-0.1%
5. Scala	5.1%	+0.1%

percentages of sites

Most popular client-side programming languages

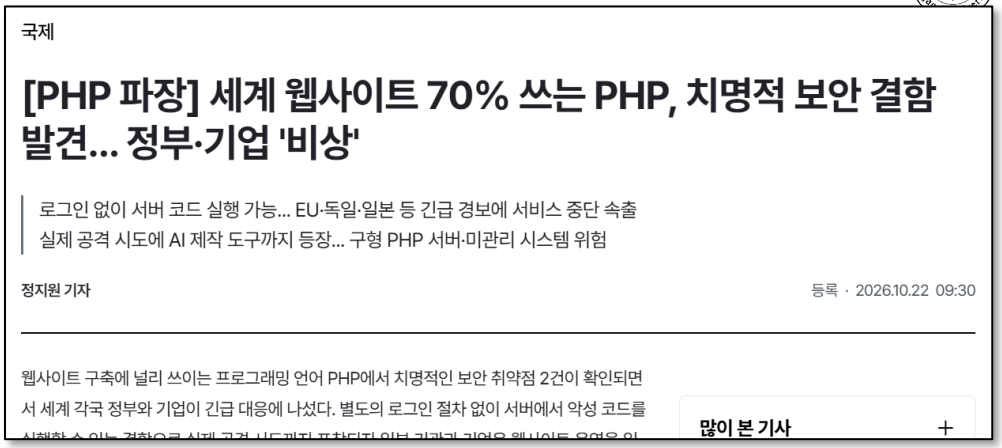
© W3Techs.com	usage	change since 1 July 2026
1. JavaScript	98.9%	
2. Flash	2.8%	

percentages of sites

Most popular JavaScript libraries

© W3Techs.com	usage	change since 1 July 2026	market share	change since 1 July 2026
1. jQuery	66.5%	-1.5%	86.5%	-0.8%
2. Bootstrap	15.1%	-0.4%	19.7%	-0.2%
3. Underscore	7.1%	-0.2%	9.3%	-0.1%
4. React	6.1%		7.9%	+0.1%
5. Modernizr	5.2%	-0.2%	6.7%	-0.2%

percentages of sites



국제

[PHP 파장] 세계 웹사이트 70% 쓰는 PHP, 치명적 보안 결함 발견... 정부·기업 '비상'

로그인 없이 서버 코드 실행 가능... EU·독일·일본 등 긴급 경보에 서비스 중단 속출
실제 공격 시도에 AI 제작 도구까지 등장... 구형 PHP 서버·미관리 시스템 위험

정지원 기자 등록 · 2026.10.22 09:30

웹사이트 구축에 널리 쓰이는 프로그래밍 언어 PHP에서 치명적인 보안 취약점 2건이 확인되면서 세계 각국 정부와 기업이 긴급 대응에 나섰다. 별도의 로그인 절차 없이 서버에서 악성 코드를

많이 본 기사 +

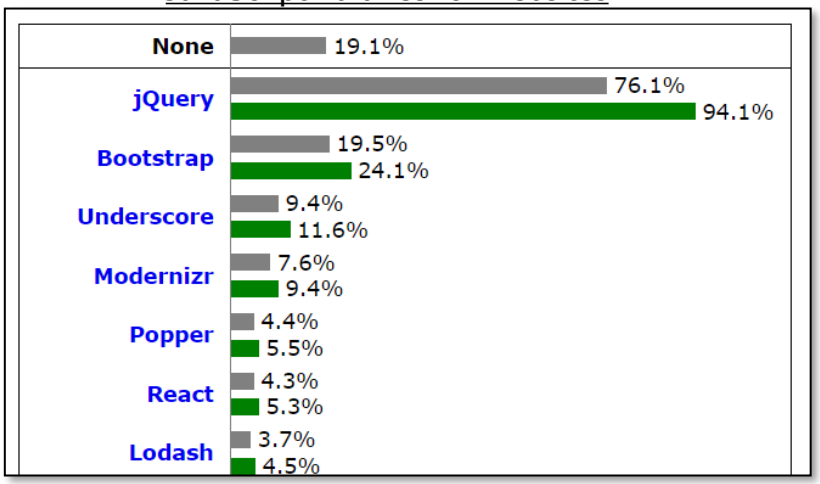
https://www.boannews.com/news/articleView.html?idxno=146621

Introduction

- Why ?
  - Javascript has become essential to current web page development, but …
  - Javascript is not a good language design
  - Javascript has become bloated (1. 부은, 부푼 2. 배가 터질 듯한)
    - ⇔ DOM navigation
    - ⇔ Browser differences
  - Writing Javascript code is tedious, time-consuming, and error-prone

- jQuery makes writing Javascript much easier
  - DOM navigation (css-like syntax)
  - Apply methods to sets of DOM elements
  - Builder model (chain method calls)
  - Extensible and there are tons of libraries
  - Handles most browser differences so you don't have to

JavaScript libraries for websites



- *Server provides data (by JSON)* and *jQuery on client provides presentation*

- **Example**

Element 찾기

```
var nodes = document.getElementsByTagName('a');
for (var i = 0; i < nodes.length; i++) {
    var a = nodes[i];
    a.addEventListener('mouseover', function(event) { event.target.style.backgroundColor='orange'; }, false );
    a.addEventListener('mouseout', function(event) { event.target.style.backgroundColor='white'; }, false );
}
```

iteration is hidden

Event

events are hidden

you let them write this, abstracting from iteration and events

```
jQuery('a').hover( function() { jQuery(this).css('background-color', 'orange'); },
                   function() { jQuery(this).css('background-color', 'white'); } );
```

01. What Is jQuery?

jQuery is a JavaScript library that runs in the browser.
It is **not a new programming** language, but a library written in JavaScript that provides a simpler way to work with the browser.

Why Was jQuery Created?

1 Repetitive DOM Code

Selecting elements and manipulating them with plain JavaScript required long and repetitive code.

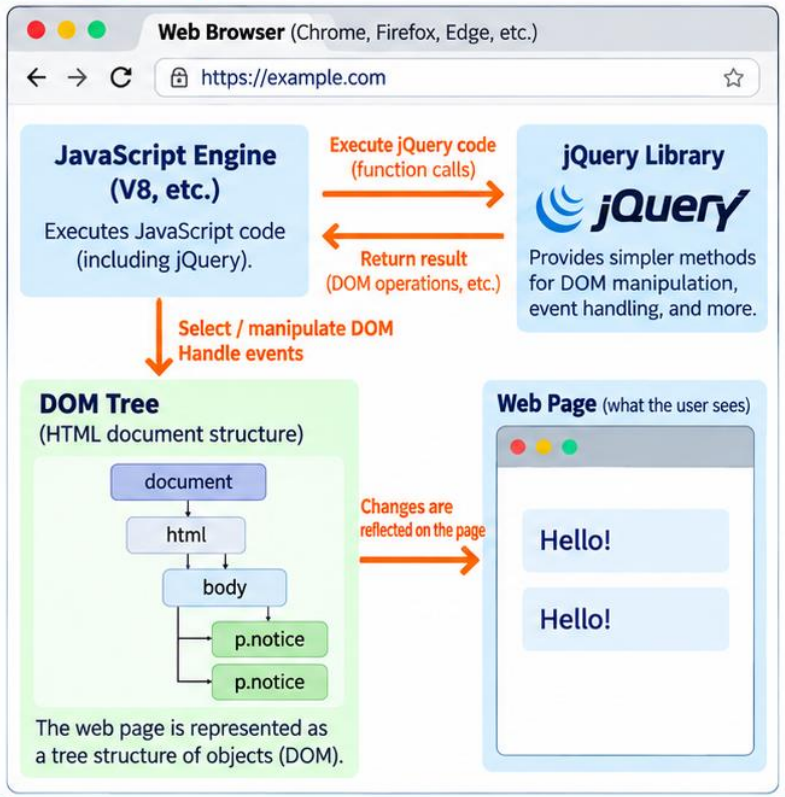
2 Browser Differences

Different browsers had **different** DOM APIs and behaviors, so developers had to write different code and handle compatibility.

3 Complex Event Handling

Handling events such as clicks, mouse, and keyboard often required more complicated code.

How jQuery Works in the Browser



Implement the Same Function in Different Ways

1 HTML (initial state)

HTML source code

```html
<body>
  <p class="notice">Message 1</p>
  <p class="notice">Message 2</p>
</body>
```

2 Vanilla JavaScript (Run after DOM ready)

Run after DOM ready

```javascript
// Change the text of all elements with class 'notice' to 'Hello!'
document.querySelectorAll('.notice').forEach(el => {
  el.textContent = 'Hello!';
});
```

3 jQuery (Run after DOM ready)

Run after DOM ready

```javascript
// Change the text of all elements with class 'notice' to 'Hello!'
$('.notice').text('Hello!');
```

4 Result (in the browser)



Modern JavaScript can also do the same tasks.

- Most of the features provided by jQuery can now be implemented with modern, standard JavaScript (ES6+).
- However, jQuery is still used and maintained in many web projects, so understanding its structure and usage is helpful in practice.

02. Loading jQuery and Running Your First Code

jQuery is provided as a pre-built library (jquery.min.js). Load it in your HTML page, and then you can use jQuery after the DOM is ready.

1. Project Folder Structure

myweb/ (project folder)

- index.html
- jquery.min.js

2. index.html Source Code

HTML File (index.html)

```
1  <p id="msg">Waiting...</p>
2
3  <script src="jquery.min.js"></script>
4  <script>
5    $(function () {
6      $('#msg').text('jQuery is ready!');
7    });
8  </script>
```

- Line 1: HTML element with initial text
- Lines 3–4: Load the jQuery library (jquery.min.js)
- Lines 5–7: jQuery code to run after the DOM is ready

4. Result in the Browser

Initial page (before script executes)

index.html

Waiting...

After jQuery executes (text changed)

index.html

jQuery is ready!

3. Code Explanation

1. `<p id="msg">Waiting...</p>`
   The p element with id="msg" shows the initial text "Waiting...".
2. `<script src="jquery.min.js"></script>`
   Load the jQuery library file (jquery.min.js) from the same folder (e.g., full build).
3. `$(function () { ... });`
   Register a function to run after the **DOM is ready**.
   This is a shorthand for $(document).ready(...).
   This point is when the DOM is ready, **not** when images are fully loaded.
4. `$('#msg').text('jQuery is ready!');`
   Change the text of the element with id="msg" to "jQuery is ready!".

5. How the Browser Processes the Page

1. **HTML Parsing (Build DOM)**

index.html (during parsing)

```
<p id="msg">Waiting...</p>
<script src="jquery.min.js">
```

DOM Tree (partial)

- document
  - html
    - body
      - p#msg "Waiting..."

2. **jQuery Library Loaded**

JS jquery.min.js → jQuery library is executed → $ is now available.

3. **DOM Ready (DOM manipulation possible)**

DOM Tree

- document
  - html
    - body
      - p#msg "Waiting..."

The DOM is ready, but images and other resources may still be loading. This is the earliest time you can safely manipulate the DOM.

4. **Run the Registered Callback Function**

`$(function () { ... })` → jQuery runs this function when the DOM is ready.

5. **DOM Update (Text Change)**

p#msg "jQuery is ready!" ← `$('#msg').text('jQuery is ready!');` changes the text of the element.

Important: Use $ after loading the library

```
<script>
  $('.error').text('Error'); // ✕ jQuery not loaded yet
</script>
```

You must load the jQuery library first, and then use $.

Reference: Run code without the ready wrapper

The following code should run after the DOM is ready.

```
$('#msg').text('jQuery is ready!');
```

Run after DOM ready

jQuery • Undergraduate Web Programming

02/10

jQuery02.html

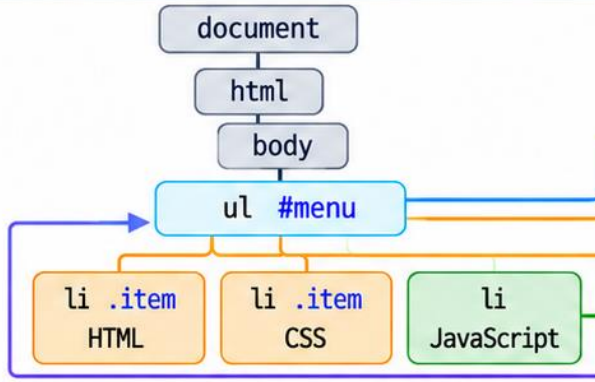
03. Selectors: Which Elements Should Change?

Use jQuery's $('selector').method() to select elements and manipulate them. The result of $() is a jQuery object that wraps a collection of DOM elements, which may contain multiple elements.

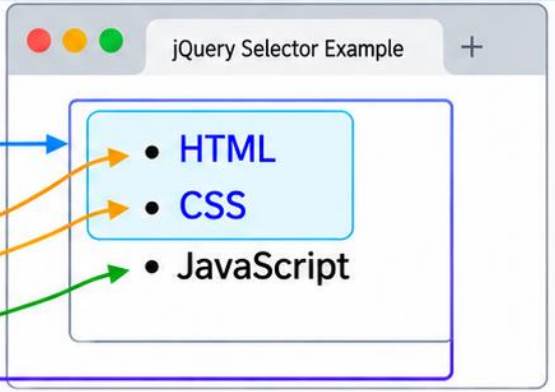
HTML Source Code

```
<ul id="menu">
   <li class="item">HTML</li>
   <li class="item">CSS</li>
   <li>JavaScript</li>
</ul>
```

DOM Tree (Structure and Selection Range)



Browser Rendering Result (Page)



Common Selectors and Their Meanings

ID selector (single element)	$('#menu')	Selects the single element whose id is menu.
Class selector (multiple elements)	$('.item')	Selects all elements with the class item. (the first two elements)
Tag selector (multiple elements)	$('li')	Selects all elements. (all three)
Descendant selector (multiple elements)	$('#menu li')	Selects all elements that are descendants of #menu. (all three)

Example: Change Element Styles

Run after DOM ready

```
// Change the text color to blue for elements with class item
$('.item').css('color', 'blue');
```

Before (initial page)




```
$('.item').
css('color', 'blue');
```

Changes the text color of elements with class item to blue.

After (modified page)



04. Reading and Writing Content, Values, and Attributes

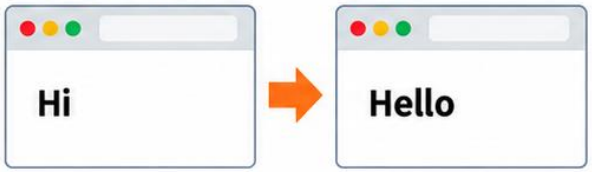
Rule If you call these methods without an argument, they return the current value. If you provide an argument, they set the value.

1. text() — Element text content

Example HTML

```
<p id="msg">Hi</p>
```

Before (in browser) → **After**



jQuery code (Run after DOM ready) // Method: .text()

```
// 1-1. Read current text (getter)
$('#msg').text();   // → 'Hi'

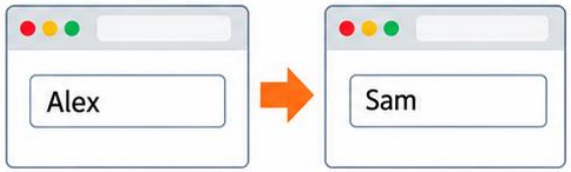
// 1-2. Change text (setter)
$('#msg').text('Hello');
```

2. val() — Form input value

Example HTML

```
<input id="name" value="Alex">
```

Before (in browser) → **After**



jQuery code (Run after DOM ready) // Method: .val()

```
// 2-1. Read current value (getter)
$('#name').val();    // → 'Alex'

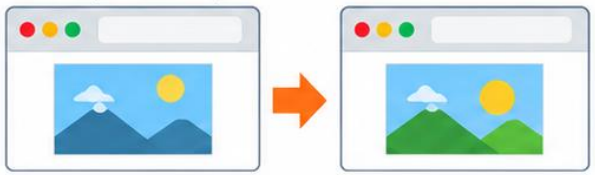
// 2-2. Change value (setter)
$('#name').val('Sam');
```

3. attr() — Element attribute value

Example HTML

```
<img id="photo" src="old.png" alt="Photo">
```

Before (in browser) → **After**



jQuery code (Run after DOM ready) // Method: .attr()

```
// 3-1. Read attribute value (getter)
$('#photo').attr('src');  // → 'old.png'

// 3-2. Change attribute value (setter)
$('#photo').attr('src', 'new.png");
```

Note .attr() requires the attribute name (e.g., 'src'). There is no no-argument getter for .attr().

.html() and .text(): Key Difference

`.html()` interprets a string as HTML and inserts it as element content. `.text()` treats the string as plain text and displays it literally. Use `.text()` for user-provided text.

```
$('#msg').html('<strong>Bold</strong>');
```



HTML is interpreted and rendered in bold.

```
$('#msg').text('<strong>Bold</strong>');
```



Tags are shown as plain text.

05. Changing Styles and Chaining Methods

05/10

Use jQuery's selection to apply classes and styles to elements, and change their content.
jQuery methods return the jQuery object, so multiple actions can be chained together.

1. Basic HTML and CSS

HTML (example code)

```
<p id="msg">Important notice</p>
```

CSS (example code)

```
.highlight {
    background: #fff3b0;
    font-weight: bold;
}
```

2. Change style and content using method chaining (Run after DOM ready)

Main code (Run after DOM ready)

```
1  $('#msg')
2      .addClass('highlight')
3      .css('color', 'navy')
4      .text('Updated!');
```

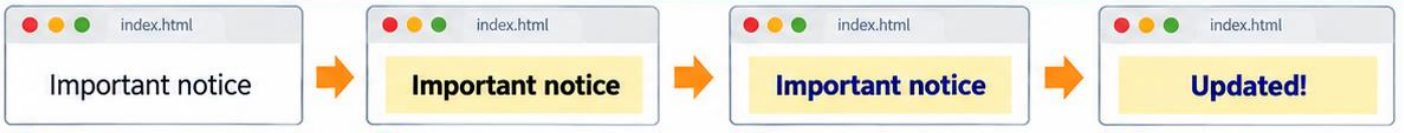
jQuery methods return the jQuery object, so you can chain multiple methods together. This is called **method chaining**.

Important notice

.text( ) used as a **getter** returns the element's text content, which is a string, not a jQuery object, so it cannot be chained.

```
$('#msg').text();
// Returns a string (chaining not allowed)
```

3. Visual result of each chained method



1. `$('#msg')` — Select the #msg element (jQuery object)
2. `.addClass('highlight')` — The highlight class is added, changing the background to yellow and the text to bold.
3. `.css('color', 'navy')` — The text color is changed to navy.
4. `.text('Updated!')` — The content is changed to 'Updated!'.

4. Adding, removing and toggling classes (Independent examples, run after DOM ready)

Initial state



Remove class

```
$('#msg').removeClass('highlight');
```



Toggle class (add/remove)

```
$('#msg').toggleClass('highlight');
```



Before toggle (class not present) — After toggle (class present)

5. Manage styles using CSS classes

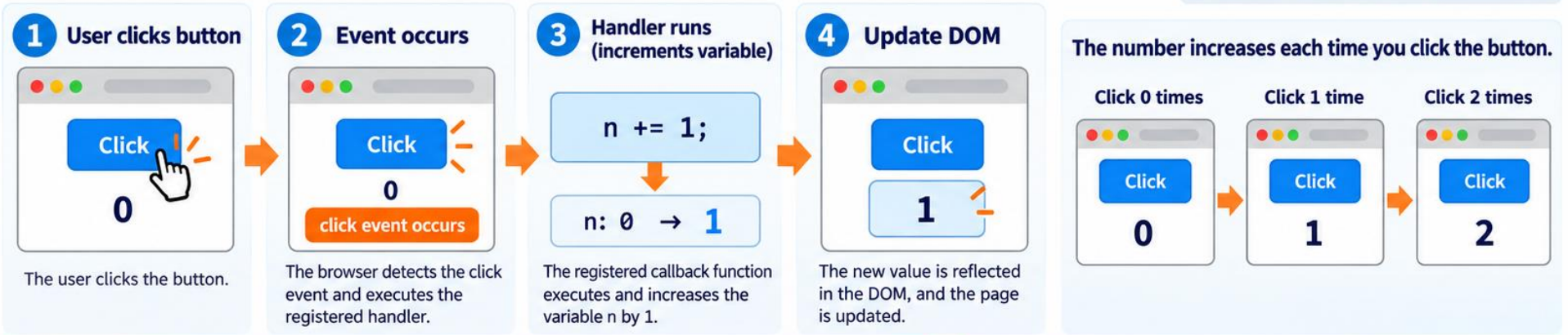
It is efficient to define frequently used styles in a CSS class and add, remove, or toggle the class when needed.

```
.highlight {
    background: #fff3b0;
    font-weight: bold;
}
```

- Can be reused for many elements
- Only the class needs to be added/removed via JavaScript
- Clearly separates style and behavior, which is easier to maintain

06. Events: Responding to User Actions

When the user performs an action such as a click or input, a previously registered callback function is executed, which can dynamically change the page.



Complete Example (HTML + jQuery)

HTML (page structure) HTML

```html
<button id="countBtn">Click</button>
<p id="count">0</p>
```

JavaScript (event registration and handling) JS

```js
$(function () {
    let n = 0;
    $('#countBtn').on('click', function () {
        n += 1;
        $('#count').text(n);
    });
});
```

Code Explanation

1. After the DOM is ready, initialize the variable n to 0.
2. Register a `click` event handler on `#countBtn`.
3. Each time the button is clicked, the callback function executes and increments n by 1.
4. Update the text of the `#count` element with n using `.text(n)`.

Important!

Registering the event handler does not execute the callback function immediately. The callback runs later when the user action (e.g., a click) actually occurs.

.on('eventName', handler) Structure

```
$('#countBtn').on('click', function () { ... });
```

- `'#countBtn'` — Element to attach the event (selector)
- `'click'` — Event type (e.g., 'click')
- `function () { ... }` — Callback function (executed when user action occurs)

Commonly Used Events (Examples)

click	input	change	submit
(when an element is clicked)	(when the user types)	(when a change is committed)	(when a form is submitted)
Click	hello	Apple	Submit

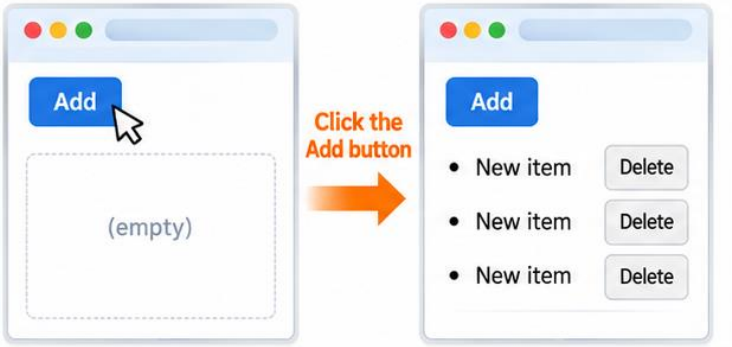
jQuery • Undergraduate Web Programming 06/10

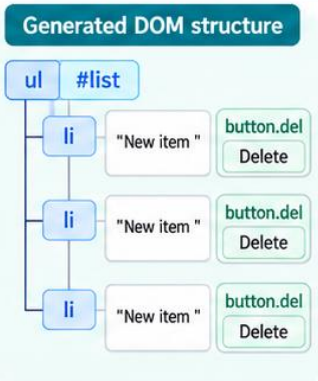
jQuery06.html

07. Adding and Removing DOM Elements: Event Delegation

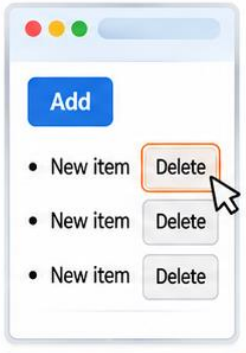
Learn how event delegation handles events on dynamically added elements.

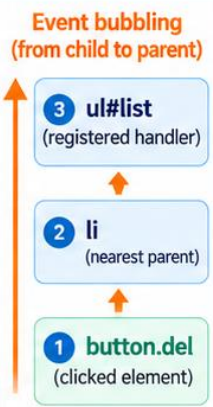
1. Add items to a list using a button click





2. Delete items using a button click and event delegation





How event delegation works

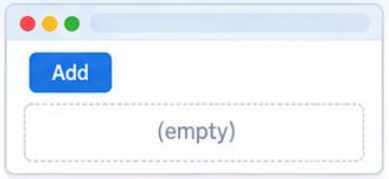
1. When a Delete button (.del) is clicked, a click event is fired.
2. The event bubbles up from the button to its li parent, and then to ul#list.
3. The handler registered on ul#list receives the event and checks if the event target matches .del.
4. If $(this).closest('li') finds a nearby li, only that item is removed.

- The parent element (#list) exists when the handler is registered.
- Later added .del buttons also work with the same handler.

HTML (initial structure) HTML

```
<button id="add">Add</button>
<ul id="list"></ul>
```

Browser view (initial state)



JavaScript (Run after DOM ready) JS

Run after DOM ready)

```
1  $('#add').on('click', function () {
2    const $li = $('<li>').text('New item ');
3    $li.append($('<button>').addClass('del').text('Delete'));
4    $('#list').append($li);
5  });
6
7  $("#list").on('click', '.del', function () {
8    $(this).closest('li').remove();
9  });
```

Code explanation

1. `$('#add').on('click', function () { ... })`
   - When the Add button is clicked, create a new li element, add a Delete button (.del) inside it, and append it to #list.
2. `$('#list').on('click', '.del', function () { ... })`
   - Register a click handler on #list, with the selector '.del'.
   - $(this) is the clicked Delete button; .closest('li') finds the nearest li and removes it.
3. Thanks to event delegation, the same handler also works for Delete buttons added later, without registering additional handlers.

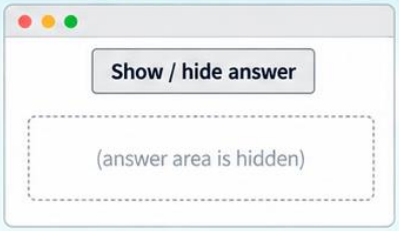
jQuery • Undergraduate Web Programming 07/10

jQuery07.html

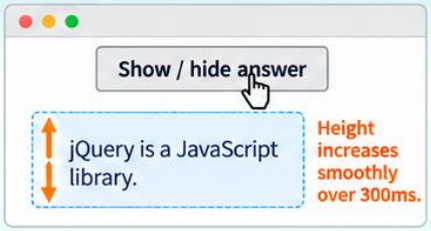
08. Visual Effects: Show, Hide, and Slide

jQuery's effect methods make elements visible, hidden, or slide smoothly. Widely used for FAQs and interactive content.

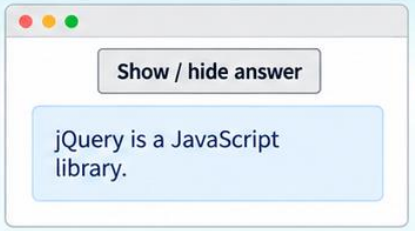
1 Initial state (hidden)
The answer is hidden when the page loads.



2 After click (300ms animation)
Height changes from 0 to the actual height over 300ms.



3 Answer fully shown
After 300ms, the answer is fully visible.



4 Click again (collapse)
Clicking again collapses the height over 300ms.



HTML (structure)
Button and answer area.

```
1  <button id="toggle">Show / hide answer</button>
2  <div id="answer">jQuery is a JavaScript library.</div>
```

CSS (initial state)
Hide the answer area at first.

```
1  #answer {
2      display: none;
3  }
```

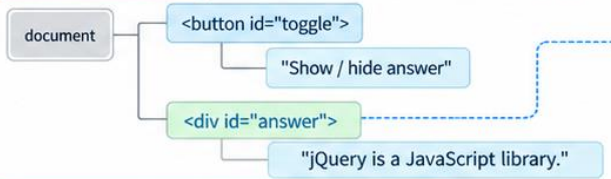
jQuery (effect execution)
Run after DOM ready. Click the button to toggle with a slide effect.

```
1  $('#toggle').on('click', function () {
2      $('#answer").stop(true, true).slideToggle(300);
3  });
```

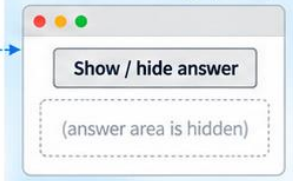
About .stop(true, true)
Clears any queued effects and completes the current one immediately. Prevents multiple queued animations when the button is clicked repeatedly.



DOM structure
Initially, the #answer element has display: none.



Browser rendering (initial)

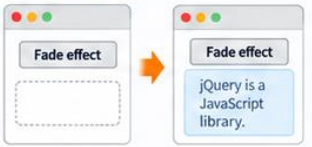


Comparison of main effect methods
Different methods can show/hide elements in various ways.

show() / hide()
Show or hide immediately (no animation).



fadeToggle(300)
Fade in/out over 300ms (changes opacity).



slideToggle(300)
Slide vertically over 300ms (changes height, this example).



Note on jQuery versions
Effect methods (show, hide, fadeToggle, slideToggle, etc.) are included in the full build of jQuery. The slim build does not include effects.

jQuery • Undergraduate Web Programming

08/10

jQuery08.html

09. Ajax: Fetching Server Data Without Reloading

09/10

With Ajax, you can request JSON data from the server and update only the necessary part of the page without reloading the entire page.

1. Browser (User Action)

http://localhost/student.html

Student Information

Load student

Loading...

1 When the user clicks the button, Ajax sends a request to the server. The page is not reloaded.

2. HTTP GET Request (asynchronous)

/api/student?id=7

4. JSON Response

{ "name": "Alex" }

3. Server



The server is assumed to have an endpoint /api/student?id=7, which returns the following JSON:

{ "name": "Alex" }

5. Browser (After Response)

http://localhost/student.html

Student Information

Load student

Alex

In the done callback, the data from the #result element is changed to 'Alex'. Other parts of the UI can continue to be used.

DOM Update Process

```
<p id="result"></p>
```

Click button (request in progress)

```
<p id="result">Loading...</p>
```

After response (done)

```
<p id="result">Alex</p>
```

HTML (Part of the Page)

student.html

```
<button id="load">Load student</button>
<p id="result"></p>
```

jQuery (Run after DOM ready)

Run after DOM ready

```
$('#load').on('click', function () {
  $('#result').text('Loading...');
  $.ajax({
    url: '/api/student',
    method: 'GET',
    data: { id: 7 },
    dataType: 'json'
  }).done(function (data) {
    $('#result').text(data.name);
  }).fail(function () {
    $('#result').text('Request failed');
  });
});
```

Key Points

1. When the button is clicked, $.ajax() sends an HTTP GET request to /api/student?id=7 (asynchronously).
2. The server returns { "name": "Alex" } as JSON. A full server implementation is required, and it must be on the same origin.
3. dataType: 'json' specifies the expected response format, and jQuery automatically parses the JSON.
4. On success (done), use data.name to update only the #result element's text to 'Alex'.
5. On failure (fail), display 'Request failed'. Other parts of the UI remain usable.

jQuery • Undergraduate Web Programming 09/10

jQuery09.html api/student

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

<button id="load">Load student</button>
<p id="result"></p>


<script>
  $(function () {
  $('#load').on('click', function () {
    $('#result').text('Loading...');

    $.ajax({
      url: 'cgi-bin/student.php',
      method: 'GET',
      data: { id: 7 },
      dataType: 'json'
    })
      .done(function (data) {
        $('#result').text(data.name);
      })
      .fail(function () {
        $('#result').text('Request failed');
      });
  });
});
</script>
```

cspro.sogang.ac.kr/tmp/jhnang/jQuery09.html

```php
<?php
header('Content-Type: application/json; charset=UTF-8');

// GET 요청의 id를 정수로 읽기
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);

if ($id === null || $id === false || $id <= 0) {
    http_response_code(400);

    echo json_encode([
        'error' => 'Invalid student ID'
    ]);
    exit;
}

// 실습용 데이터: 실제 서비스에서는 DB에서 조회
$students = [
    7 => ['name' => 'Alex'], 8 => ['name' => 'Sam'], 9 => ['name' => 'Chris']
];

// 해당 학생이 없으면 HTTP 404 응답
if (!isset($students[$id])) {
    http_response_code(404);
    echo json_encode([
        'error' => 'Student not found'
    ]);
    exit;
}

// 학생 정보를 JSON으로 응답
echo json_encode(
    $students[$id],
    JSON_UNESCAPED_UNICODE
);
?>
```

cspro.sogang.ac.kr/tmp/jhnang/cgi-bin/student.php

10. Putting It Together: Search Without Reloading

Use jQuery Ajax to send a request to the server, update part of the page without reloading, and display the search results.

1 User Input



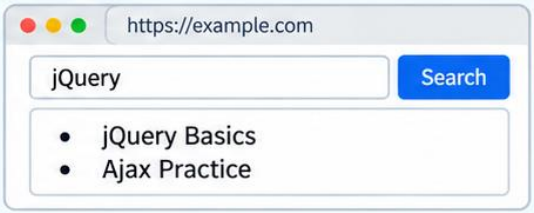
Type a search query and click the Search button.

2 Request to Server with Ajax



jQuery's `$.getJSON()` sends the query to the server and receives the results in JSON format.

3 Update Result Area (DOM Manipulation)



Use the returned data to dynamically update the `#results` list and display the results.

HTML (Search Form and Result Area)

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Ajax Search Example</title>
  <script src="jquery.min.js"></script>
</head>
<body>
  <form id="searchForm">
    <input id="q" placeholder="Search...">
    <button type="submit">Search</button>
  </form>
  <ul id="results"></ul>
</body>
</html>
```

JavaScript (Event Handling and Ajax)

Run after DOM ready | JS

```javascript
$(function () {
  $('#searchForm').on('submit', function (e) {
    e.preventDefault();
    const q = $('#q').val().trim();
    if (!q) return;
    $('#results').empty();
    $.getJSON('/api/search', { q: q })
      .done(function (items) {
        items.forEach(function (item) {
          $('<li>').text(item.title).appendTo('#results');
        });
      })
      .fail(function () {
        $('<li>').text('Search failed').appendTo('#results');
      });
  });
});
```

Example API Response (JSON)

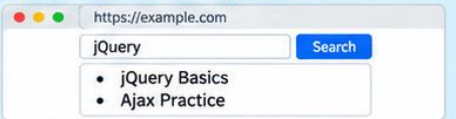
JSON

```json
[
  { "title": "jQuery Basics" },
  { "title": "Ajax Practice" }
]
```

Key Points

- `e.preventDefault()`: Prevent the form's default submission (no page reload).
- `$.getJSON()`: Send the search query and process the JSON response.
- Use the returned data to create `<li>` elements and add them to `#results`.
- Use `.text()` to safely display server-provided titles.
- The endpoint `/api/search` is assumed to exist on the same-origin server.
- Use the full build of jQuery (jquery.min.js) which includes Ajax.

Example Result (query: jQuery)



```html
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Ajax Search Example</title>

  <!-- 같은 폴더에 jQuery full build 파일을 저장 -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head> <body>
  <form id="searchForm">
    <input id="q" placeholder="Search...">
    <button type="submit">Search</button>
  </form>

  <ul id="results"></ul>

  <script>
    $(function () {
      $('#searchForm').on('submit', function (e) {
        // 기본 폼 제출에 의한 페이지 이동 방지
        e.preventDefault();

        // 입력값을 읽고 앞뒤 공백 제거
        const q = $('#q').val().trim();
        if (!q) return;

        // 이전 검색 결과 삭제
        $('#results').empty();

        // GET 요청 : api/search.php?q=검색어
        $.getJSON('cgi-bin/jQuery_search.php', { q: q })
          .done(function (items) {
            items.forEach(function (item) {
              $('<li>').text(item.title).appendTo('#results');
            });
          })
          .fail(function () {
            $('<li>').text('Search failed').appendTo('#results');
          });
      });
    });
  </script> </body> </html>
```

cspro.sogang.ac.kr/tmp/jhnang/jQuery10.html

```php
<?php
header('Content-Type: application/json; charset=UTF-8');

// GET 요청으로 받은 검색어
$q = trim($_GET['q'] ?? '');

// 실습용 데이터: 실제 서비스에서는 DB에서 검색
$documents = [
    [ 'title' => 'jQuery Basics',
      'keywords' => 'jquery javascript basics' ],
    [ 'title' => 'Ajax Practice',
      'keywords' => 'jquery ajax json' ],
    [ 'title' => 'PHP Introduction',
      'keywords' => 'php server programming' ]
];

$results = [];

if ($q !== '') {
    foreach ($documents as $document) {
        $searchText =
            $document['title'] . ' ' . $document['keywords'];

        if (stripos($searchText, $q) !== false) {
            $results[] = [
                'title' => $document['title']
            ];
        }
    }
}

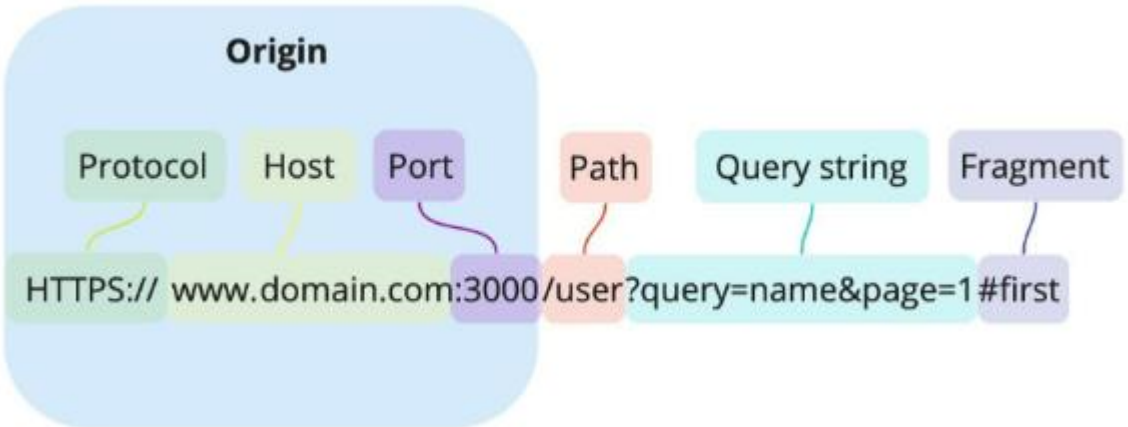
// PHP 배열을 JSON 문자열로 변환하여 응답
echo json_encode($results, JSON_UNESCAPED_UNICODE);
```

cspro.sogang.ac.kr/tmp/jhnang/cgi-bin/jQuery_search.php

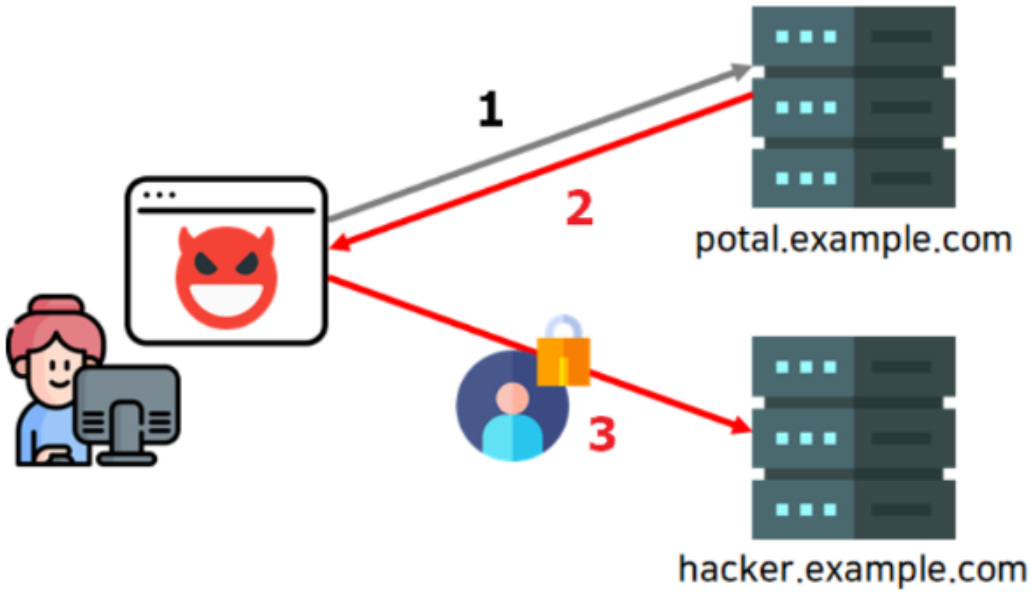
<참고> CORS (Cross-Origin-Resource-Sharing)

- **SOP (Same Origin Policy)**
  - ✓ 웹 생태계에서 다른 출처로의 리소스 요청을 제한하는 정책 중 하나로, 같은 출처의 리소스만 공유할 수 이도록 제한하는 정책
  - ✓ 현재 브라우저는 SOP (Same Origin Policy) 정책을 지키고 있는데, 이를 위반하는 요청을 하게 되면 CORS 오류가 발생하는 것이다.

- **Cross Origin Resource Sharing Error**
  - ✓ Cross-Origin이란 다음 중 한가지라도 다른 경우
    1.프로토콜 - http와 https는 프로토콜이 다르다.
    2.도메인 - domain.com과 other-domain.com은 다르다.
    3.포트 - 8080포트와 3000포트는 다르다



• Cross-Origin의 위험성



1. 사용자가 악성 사이트에 접속한다.

2. 이때 해커가 몰래 심어놓은 악의적인 자바스크립트가 실행되어, 사용자가 모르는 사이에 어느 포털 사이트에 요청을 보낸다.

3. 그럼 포털 사이트에서 해당 브라우저의 쿠키를 이용하여 로그인을 하거나 등 상호작용에 따른 개인 정보를 응답 값을 받은뒤, 사이트에서 해커 서버(hacker.example.com)로 재차 보낸다.

4. 이외에도 사용자가 접속중인 내부망의 아이피와 포트를 가져오거나, 해커가 사용자 브라우저를 프록시처럼 악용할 수도 있다.

이렇듯 악의적인 공격에서부터 웹사이트를 지키기 위해서 SOP가 존재하는 것이다.